

A **transaction in a DBMS is a single logical unit of data processing** that executes one or more database operations (like reading, inserting, updating, or deleting data) as a complete package.

Core Concept of a Transaction

Think of a transaction as an **"all-or-nothing" operation**.

For example, when you transfer \$100 from Account A to Account B, the database must perform two distinct steps:

1. Deduct \$100 from Account A.
2. Add \$100 to Account B.

If the system crashes after step 1 but before step 2, money disappears. A transaction ensures that if any single step fails, the entire process is canceled (**rolled back**), and the database returns to its original state. If all steps succeed, changes are permanently saved (**committed**).

The ACID Properties

To maintain data integrity and consistency, every transaction in a DBMS must strictly follow the **ACID** properties:

1. Atomicity ("All or Nothing")

- **Definition:** A transaction is treated as a single, indivisible unit of work.
- **Mechanism:** Either all operations succeed, or none of them do. If a failure occurs midway, the database reverts all changes made by that transaction.

2. Consistency (Preserving Rules)

- **Definition:** A transaction must transform the database from one valid, consistent state to another valid, consistent state.
- **Mechanism:** It must satisfy all database constraints, rules, and triggers (e.g., account balances cannot drop below zero).

3. Isolation (Independent Execution)

- **Definition:** Multiple transactions executing at the same time must not interfere with each other.
- **Mechanism:** The intermediate state of a running transaction is invisible to other concurrent transactions. It behaves as if it is the only transaction running in the system.

4. Durability (Permanent Changes)

- **Definition:** Once a transaction is successfully completed (committed), its changes are permanent.
 - **Mechanism:** The updates will survive any subsequent system failures, power outages, or crashes, usually because they are written to non-volatile storage (like a hard drive).
-

Transaction States

During its lifecycle, a transaction moves through several distinct states:

- **Active:** The initial state where the transaction execution begins and operations are being performed.
 - **Partially Committed:** The state after the final operation has been executed, but before changes are safely written to disk.
 - **Committed:** The state after successful completion; changes are now permanent.
 - **Failed:** The state reached if any operation fails or if the transaction is aborted while active.
 - **Aborted:** The state after the transaction is rolled back and the database is restored to its pre-transaction condition.
-

Transaction Control Commands

In SQL databases, transactions are managed using specific Control Commands (TCL):

- `BEGIN TRANSACTION` : Marks the starting point of a transaction.
 - `COMMIT` : Saves all changes permanently to the database.
 - `ROLLBACK` : Undoes all changes if an error occurs during execution.
 - `SAVEPOINT` : Creates temporary checkpoints within a transaction so you can roll back to a specific point rather than the very beginning.
-

DBMS Transactions Lab Manual

A transaction is a single unit of program execution that accesses and possibly updates various data items. Transactions must follow the **ACID** properties: Atomicity, Consistency, Isolation, and Durability. [1, 2, 3, 4, 5]

Objective

- Understand transaction control language (TCL) commands: `COMMIT` and `ROLLBACK`.
- Implement transaction handling using InnoDB storage engine in XAMPP MySQL. [6, 7, 8, 9]

Step 1: Environment Setup in XAMPP

By default, MySQL automatically saves every change you make. To practice transactions, you must turn off this feature. [10, 11]

1. Open **XAMPP Control Panel** and start **Apache** and **MySQL**.
2. Click **Admin** next to MySQL to open **phpMyAdmin** (or use the MySQL CLI).
3. Click the **SQL** tab.
4. Run this command to disable automatic saving for your current session:
5. `SET autocommit = 0;`

[12, 13, 14, 15, 16]

Step 2: Create a Sample Database and Table

Copy and run the following script to create a bank account table. Notice the `ENGINE=InnoDB` clause; transactions do not work on older `MyISAM` tables. [17, 18]

```
CREATE DATABASE LabTransactions;
USE LabTransactions;

CREATE TABLE Accounts (
    account_id INT PRIMARY KEY,
    name VARCHAR(50),
    balance DECIMAL(10,2)
) ENGINE=
```

[InnoDB](#)

```
;
```

```
INSERT INTO Accounts VALUES (1, 'Alice', 1000.00);
INSERT INTO Accounts VALUES (2, 'Bob', 500.00);
[19]
```

Step 3: Exercise 1 — Successful Transaction (COMMIT)

This scenario simulates a successful money transfer of \$200 from Alice to Bob.

SQL Commands:

```
-- 1. Start the transaction
START TRANSACTION;

-- 2. Deduct money from Alice
UPDATE Accounts
SET balance = balance - 200.00
WHERE account_id = 1;

-- 3. Add money to Bob
UPDATE Accounts
SET balance = balance + 200.00
WHERE account_id = 2;

-- 4. Permanently save the changes
COMMIT;
```

Verification:

Run a selection query to check the updated balances.

```
SELECT * FROM Accounts;
```

- **Expected Result:** Alice has 800.00, Bob has 700.00.
-

Step 4: Exercise 2 — Failed Transaction (ROLLBACK)

This scenario simulates an error during a transfer. Alice tries to send \$300 to Bob, but the system crashes before Bob receives it. We must undo the deduction. [20, 21, 22]

SQL Commands:

```
-- 1. Start a new transaction
START TRANSACTION;

-- 2. Deduct money from Alice
UPDATE Accounts
SET balance = balance - 300.00
WHERE account_id = 1;
```

```
-- 3. SIMULATED ERROR/FAILURE OCCURS HERE
-- (We decide to cancel the operation because of the error)

-- 4. Undo all changes made since START TRANSACTION
ROLLBACK;
```

Verification:

Run a selection query to check the balances.

```
SELECT * FROM Accounts;
```

- **Expected Result:** Alice stays at 800.00, Bob stays at 700.00. The deduction was successfully undone.
-

Step 5: Exercise 3 — Partial Rollback (SAVEPOINT)

Savepoints allow you to roll back a part of a transaction instead of the whole thing. [23]

SQL Commands:

```
START TRANSACTION;

-- Update 1: Give Alice a bonus
UPDATE Accounts SET balance = balance + 100.00 WHERE account_id = 1;

-- Create a landmark checkpoint
SAVEPOINT BonusAdded;

-- Update 2: Give Bob a bonus by mistake
UPDATE Accounts SET balance = balance + 500.00 WHERE account_id = 2;

-- Undo only the mistake (Update 2)
ROLLBACK TO SAVEPOINT BonusAdded;

-- Save the rest (Update 1)
COMMIT;
```

Verification:

```
SELECT * FROM Accounts;
```

- **Expected Result:** Alice has 900.00 (bonus kept). Bob has 700.00 (mistake undone).